ERI Data Signature process guide

(v0.2)

1) Sample implementation to generate signed data

Please follow the below mentioned code snap to generate the signature data by using the private key.

Step 1: sign plain text
Step 2: convert signed data into bytes
Step 3: convert bytes in to base64 to send as payload

Imports:

```
import java.io.File;
import java.io.FileInputStream;
import java.io.IOException;
import java.security.KeyStore;
import java.security.PrivateKey;
import java.security.Provider;
import java.security.Security;
import java.security.cert.Certificate;
import java.security.cert.CertificateExpiredException;
import java.security.cert.CertificateNotYetValidException;
import java.security.cert.X509Certificate;
import java.util.ArrayList;
import java.util.Collection;
import java.util.List;

import org.bouncycastle.cert.X509CertificateHolder;
import org.bouncycastle.cert.jcajce.JcaCertStore;
import org.bouncycastle.cms.CMSException;
import org.bouncycastle.cms.CMSProcessableByteArray;
import org.bouncycastle.cms.CMSSignedData;
import org.bouncycastle.cms.CMSSignedDataGenerator;
import org.bouncycastle.cms.CMSTypedData;
import org.bouncycastle.cms.SignerInformation;
import org.bouncycastle.cms.SignerInformationStore;
import org.bouncycastle.cms.jcajce.JcaSignerInfoGeneratorBuilder;
import org.bouncycastle.cms.jcajce.JcaSimpleSignerInfoVerifierBuilder;
import org.bouncycastle.jce.provider.BouncyCastleProvider;
import org.bouncycastle.operator.ContentSigner;
import org.bouncycastle.operator.OperatorCreationException;
import org.bouncycastle.operator.jcajce.JcaContentSignerBuilder;
import org.bouncycastle.operator.jcajce.JcaDigestCalculatorProviderBuilder;
import org.bouncycastle.util.Store;
import org.bouncycastle.util.encoders.Base64;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public static byte[] generateSign(String data, String eriId) throws Exception {
                System.out.println("Entering generateSign at " +
                System.currentTimeMillis());

                String pfxCred = "";
                String pfxType = "";
                String alias = "";
                Certificate[] certChain = null;
                List<Certificate> certList = null;
```

```
JcaCertStore certStore = null;
PrivateKey privKey = null;
X509Certificate certificate = null;
try {
Security.addProvider((Provider) new BouncyCastleProvider());

String certFilePath = "<private key path>";
pfxType = "PKCS12";
pfxCred = "<keystore pass>";
alias = "agencykey";
KeyStore keyStore = KeyStore.getInstance(pfxType, "BC");
keyStore.load(new FileInputStream(new File(certFilePath)),
pfxCred.toCharArray());
certChain = keyStore.getCertificateChain(alias);
certList = new ArrayList<>();
for (int i = 0; i < certChain.length; i++)
certList.add(certChain[i]);
certStore = new JcaCertStore(certList);
KeyStore.PrivateKeyEntry entry = (KeyStore.PrivateKeyEntry)
keyStore.getEntry(alias,
new KeyStore.PasswordProtection(pfxCred.toCharArray()));
privKey = entry.getPrivateKey();
certificate = (X509Certificate) keyStore.getCertificate(alias);
X509CertificateHolder certificateHolder = new
X509CertificateHolder(certificate.getEncoded());
CMSSignedDataGenerator cmsSignedDataGenerator = new
CMSSignedDataGenerator();
ContentSigner sha1Signer = (new
JcaContentSignerBuilder("SHA256withRSA")).setProvider("BC").build(privKe
y);
cmsSignedDataGenerator.addSignerInfoGenerator((new
JcaSignerInfoGeneratorBuilder(
(new
JcaDigestCalculatorProviderBuilder()).setProvider("BC").build())).build(
sha1Signer,
certificateHolder));
cmsSignedDataGenerator.addCertificates((Store) certStore);
CMSProcessableByteArray cMSProcessableByteArray = new
CMSProcessableByteArray(data.getBytes());
CMSSignedData sigData = cmsSignedDataGenerator.generate((CMSTypedData)
cMSProcessableByteArray, false);
       return sigData.getEncoded();

} catch (Exception e) {
              e.printStackTrace();
              return null;
       } finally {
              LOGGER.info("Exit generateSign at " +
System.currentTimeMillis());
       }
}
```

Payload Sample:-

```
SignedDataResponseDto response = new SignedDataResponseDto();
response.setSign(Base64.encodeBase64String(signedData));
response.setData(Base64.encodeBase64String(dataToSigned.getBytes()));

response.setEriUserId(eriId);
```

"signedData" is the data generated from above code snap.

"dataToSign" is the original text.

Final payload output sample example: -


```
{
    "sign": "MIAGCSqGSIb3DQEHAqCAMIACAQExDzANBglghkgBZQMEAgEFADCABgkqhkiG9w0BBwEAAKCAM
IICzzCCAbegAwIBAgIESFrftjANBgkqhkiG9w0BAQUFADAYMRYwFAYDVQQDEw1jb21tb25zZXJ2aWNlMB4XDTI
wMTEyNjA3MDUxMVoXDTIyMDcxOTA3MDUxMVowGDEWMBQGA1UEAxMNY29tbW9uc2VydmljZTCCASIwDQYJKoZIh
vcNAQEBBQADggEPADCCAQoCggEBALj29rfHjusMPy7HhDh1MdM6JQlhutCndh/q4haSgeym9LelLchQGUWsRk2
yPPszlZ3R6YiUPcGojJDUCw3wQh1aHLlRveYqdbtCYflL3PynRa/pdFMukJmpa2h3+Dk3el0f/1VHbtdozXAIz
3AWeyS33H4jxmqyFstyhcb5c+cXMsKoXNckRxBrd76JB5iNuaf2I4Xa25kgMIUJ9V2EXMlXa3vgowE1E2WR4yi
q+UXOUpc7q7GeUdxKnkoxe7h+/J6vsBE31TNC3X4+iqeUvYANqnK0YfBnhdbMVAGcpmR30mjaxw1iYR1XOGX+l
J423DTuBqref6Hg044gexhAFMkCAwEAAaMhMB8wHQYDVR0OBBYEFOP3f60GE1DlVjVL0fyHZjtOauwuMA0GCSq
GSIb3DQEBBQUAA4IBAQBnUReIWwM0QRX4/CIvi0LJ9DXmoTqnPpMMdLjwR0VwNotAWIRiVQZjb6TI+lVYbSW4c
F3FHXlwg5y1JjS4EDEZIvovVgQSD1cuSuxTYhTaRKVUWd+UeDAdIuJ19aeeYo6agtKR51aDt536Vik0LyitSO5
pWsOhKRKhVs4EdmUdEOkVxArHv2AYx4y1f/dgUWl+PYbsoqf/1pp5saeHdDlV+PzypbYs1v1kde8EGC3+npXgI
RV/vP8YVkl3CLzbHZ0bJ41gP8d1D/lRfRmsvQ+Tz89D6qR8T1BsajOgk58EOu3gyCyDEKkfkpaKkrGZrmD+Jxn
z5Q3h4vjVx72gXYdLAAAxggHmMIIB4gIBATAgMBgxFjAUBgNVBAMTDWNvbW1vbnNlcnZpY2UCBEha37YwDQYJY
IZIAWUDBAIBBQCggZgwGAYJKoZIhvcNAQkDMQsGCSqGSIb3DQEHATAcBgkqhkiG9w0BCQUxDxcNMjEwOTE2MTQ
xMDMzWjAtBgkqhkiG9w0BCTQxIDAeMA0GCWCGSAFlAwQCAQUAoQ0GCSqGSIb3DQEBCwUAMC8GCSqGSIb3DQEJB
DEiBCC9QIPPvywztGipzbNv2C0yq3TAztzoTFsBiKesNZKq9zANBgkqhkiG9w0BAQsFAASCAQCoIK43ttH+suE
cZuJy8X33CPXUwwZ0R8BwAXJsKILiDTiNGURhZQxqLakGAtgTEtLjhr5Lj/Re1OKwrPGEu3R5BUjBpw017td1D
9fw9pqm7lCIwI4wglIwF9qEdketEL+GA/61BZx2/ND6L/tpxMQ6ukZwKNUctZhUOOxsQpIdBAs4P04Tbd+I1mn
4kTjqVkhPrCX55IboH+QrpXlfaDiuXs7AR/CFF8DJzoBNSxiUDP30wdhjR+mOCTnsJeiGYuppsVRLTh9Cme+UK
167QxbcmZUXVl79ufu3vt1GAryXokNHhX7y0kGeAri+L1DmZNlo2kec6DvCuyXy/yVqMDazAAAAAAAA",
    "data": "ew0KCSJzZXJ2aWNlTmFtZSI6IkVyapZ25EYXRhU2VydmljZSIsDQogICAgImVudGl0eSI6ICJ
FUklVMDEwODg4IiwNCgkicGFzcyI6ICJPcmFjbGVAMTIzIg0KfQ==",
    "eriUserId": "ERIU010898"
}
```


<< The below is given for self-testing purpose to verify if the signed data can be properly verified, if this works then you should be able get proper response from login API>>

2) Use this method to Verify/Validate the signed data generated.

Below code can be referred to verify the payload generated

Step 1 : decode the base64 sign data and data.

Step 2: convert to bytes and verify

```
public static Boolean verifySignedData(final String signedData, final String
dataToSign, String userId)
                        throws CMSException, CertificateExpiredException,
CertificateNotYetValidException {
                Boolean result = null;
```

```
		try {
			String filePath = <public file path>
			if (filePath.isEmpty()) {
				result = false;
			} else {


				final X509Certificate x509Certificate =
getX509Certificate(filePath);
				if (x509Certificate != null) {
					x509Certificate.checkValidity();
				}
				result = verifySignedData(signedData, dataToSign,
x509Certificate);
			}

			LOGGER.info("data signature verified");
			return result;
		} catch (IllegalArgumentException e) {
			LOGGER.error("IllegalArgumentException occured in getArguments
method {}", e);
			return false;
		} catch (final Exception e) {
			LOGGER.error("Exception in signature verification", e);
			return false;
		}
	}



	private static boolean verifySignedData(final String sign, final String data,
final X509Certificate x509Certificate)
			throws CMSException, OperatorCreationException {

		Security.addProvider(new
org.bouncycastle.jce.provider.BouncyCastleProvider());


		final CMSProcessableByteArray cmsProcessableByteArray = new
CMSProcessableByteArray(
				Base64.decode(data.getBytes()));

		final CMSSignedData cms = new CMSSignedData(cmsProcessableByteArray,
		Base64.decode(sign.getBytes()));
		final SignerInformationStore signers = cms.getSignerInfos();
		final Collection<SignerInformation> c = signers.getSigners();

		for (final SignerInformation signer : c) {
			final boolean verify = signer.verify(new
			JcaSimpleSignerInfoVerifierBuilder().setProvider("BC").bui
			ld(x509Certificate));
			return verify;
		}

		return false;
	}

	private static X509Certificate getX509Certificate(final String filePath) throws
IOException {
		FileInputStream fis = null;
		try {
```

```
                        final File file = new File(filePath);
                        fis = new FileInputStream(file);
                        KeyStore ks = KeyStore.getInstance("PKCS12");
                        String pwd =  KEYSTORE PASSORD>"; //replace
                        LOGGER.debug("Keystore password fetched");

                        ks.load(fis, pwd.toCharArray());
                        Certificate cert = ks.getCertificate("agencykey");
                        return (X509Certificate) cert;
                } catch (IllegalArgumentException e) {
                        LOGGER.error("IllegalArgumentException occured in getArguments
        method {}", e);
                } catch (final Throwable th) {
                        LOGGER.error("Throwable {} ", th);
                } finally {
                        if (fis != null) {
                                fis.close();
                        }
                }
                return null;
        }
```

If the certificates are on place and signed logic is correct, it should return true. Then you can procced testing with the APIs.

<< The below is given for as a next step to encrypt the password to avoid send the clear text password, implement this is after you get go ahead from ERIhelp>>

3) **Password Encryption**

Below code snap can we use to encrypt the text

```
// for plain text to encrypted text
public String getEncryptedPlainText(String plainText, SecretKey key) throws Exception {

        cipher = Cipher.getInstance("AES");
        byte[] plainTextByte = plainText.getBytes();
 cipher.init(Cipher.ENCRYPT_MODE, key);
 byte[] encryptedByte = cipher.doFinal(plainTextByte);
 Base64.Encoder encoder = Base64.getEncoder();
 String encryptedText = encoder.encodeToString(encryptedByte);
 return encryptedText;
        }
```

After getting the base64 encrypted password, use this password in EriLoginService and then sign the data.

4) **Data Decryption**

Below code snap can we use to encrypt the text

```
// for encrypted text to plain text
public String getDecryptedPlainText(String encryptedString, SecretKey key) throws
Exception {

        cipher = Cipher.getInstance("AES");
```

```
        Base64.Decoder decoder = Base64.getDecoder();
 byte[] encryptedTextByte = decoder.decode(encryptedString);
 cipher.init(Cipher.DECRYPT_MODE, key);
 byte[] decryptedByte = cipher.doFinal(encryptedTextByte);
 String decryptedText = new String(decryptedByte);
 return decryptedText;
}
```